

UberEcho- An AI Support Assistant for Uber's Twitter Team

Executive Summary

This report describes an AI assistant I built to help Uber's support team handle customer tweets, designed with safety as the top priority — a bot handling public complaints can't promise fake refunds, ask for private details in public, or miss a real emergency.

The overall approach is based on a pattern called **OrchestRAG** (I adapted this design from an open-source project by thorOdinson16: github.com/thorOdinson16/OrchestRAG). In plain terms, it combines an instant keyword check for danger, a search step that finds similar past conversations, and a classifier that sorts messages into categories. With this setup, the system catches **100% of safety emergencies**, correctly sorts messages **71% of the time** (about 33 points better than basic keyword matching), and writes replies that score **3.80/5** for quality, closely matching a human reviewer's scores on the same replies.

1. What "Good" Actually Means for Uber's Twitter Support

1. **Public feed vs. private fix.** Anything the bot says is visible to everyone. Publicly promising "I've credited you \$50" is a real liability. A good bot shows empathy and routes people to a private, verified channel to actually resolve things.
2. **Never ask for private details in public.** Riders and drivers shouldn't post card or phone numbers publicly. The right move is pointing to in-app self-service (e.g. "lost item") or a private DM with an account email.
3. **Not all mistakes cost the same.** A wrong billing-dispute call just means a human looks at it later. Missing a real emergency (crash, assault, drunk driver) could cause serious harm — so the system is tuned to catch every possible emergency, even at the cost of some false alarms.
4. **Knowing when to stay quiet.** Over 30% of tweets at support accounts are just venting or too vague to act on. Replying with generic boilerplate can spark backlash, so when there's no strong match to a resolvable issue, the system stays silent instead of guessing.

Core rules, no exceptions: (1) Safety always comes first — any mention of violence, a crash, or assault triggers an immediate emergency response. (2) Never send dead or outdated links — only current in-app steps or a DM request. (3) Stay quiet when unsure, rather than guessing publicly.

2. How the System Works, Step by Step

Step	What It Does	Speed / Purpose
1. Instant Safety Check	Scans for words like "police," "accident," "crash," "assault," "drunk"	Near-instant. Flags urgent cases before anything else runs.
2. Find Similar Past Cases	Searches past conversations for the closest matches by meaning, not just wording	~8ms. Finds the top 2 matches; if nothing's similar enough, the system won't reply publicly.
3. Sort Into a Category	A simple classifier picks one of 6 categories and a confidence score	~1ms. Only confident (60%+) cases get auto-handled; the rest go to a human.
4. Write the Reply	Uses an AI model (OpenAI gpt-4o-mini) to draft a reply grounded in similar past cases	Writes a short, empathetic reply under 280 characters, or a suggested draft for human review.

How messages get routed, always with a logged reason:

Bucket	When It Happens	System Action
Urgent (Po)	Safety keyword hit, OR classifier flags it as safety-related	Immediate safety message; handed to a human safety team.
Auto-Resolved	Close match to a past case ($\geq 45\%$ similar) AND classifier confident ($\geq 60\%$)	AI writes a grounded reply and posts it.
Needs Human Review (P2)	Classifier unsure, or a legal threat is mentioned	Sent to a human's inbox with an AI-drafted reply ready to approve.
Stay Silent	No close-enough match to any past case	No public reply — avoids posting something wrong to an ungrounded rant.

3. How Well Does It Actually Work?

How I Built and Labeled the Test Set

To keep the test fair, all 180 examples came only from data the system had never seen. I split the raw data by full conversation thread (not by individual tweet) into training, validation, and a separate held-out 15% test set — this stops one conversation from leaking into both sides. Every Golden Set case came from that held-out slice only.

Real safety emergencies are rare — under 0.2% of all tweets — while billing questions make up over half. A plain random sample of 180 tweets would likely contain zero or one real emergency, making it impossible to actually measure how well the system catches them. So instead, I deliberately picked 30 examples for each of the 6 categories (180 total), including 30 cases that genuinely needed to be escalated to a human.

Labeling itself was a two-step process. First, simple keyword rules sorted tweets into rough starting buckets. Then I went through every tweet myself, checked whether the keyword guess was right, and corrected it where it wasn't — keyword matching alone gets fooled easily. Two examples:

- A tweet about a driver flipping someone off after canceling — while the customer was "on the phone" trying to reach them — got auto-tagged as a lost phone, just because it contained the word "phone." I corrected it to a driver-conduct complaint, since that's what it actually was.
- A tweet mentioning "hospital" while asking about promo codes got auto-flagged as a safety emergency because of that one word. I corrected it to an account/promo issue and marked it as not needing escalation, since there was no active emergency.

The Results

Tested against two simpler approaches, using those 180 hand-labeled messages, spread evenly across all 6 categories:

System	Got Category Right	Balanced Score	Flagged Reviews Correctly	Caught Every Safety Case	Reply Quality (/5)
Basic Keyword Matching	48.3%	0.45	0%	0%	2.00
Simple Search + Keywords	38.3%	0.36	100%	50%	2.78
This System	71.1%	0.71	29.8%	100%	3.80

Triage breakdown: Auto-Resolved: 86 cases (47.8%). Needs Human Review: 67 (37.2%).

Urgent/Safety: 27 (15.0%).

Checking the AI's judgment against a human's: I had an AI reviewer score 15 sample replies for empathy, accuracy, and how actionable they were. This system scored 3.80/5, clearly ahead of the simple search approach's 2.78/5 (which lost points for sounding robotic and linking to dead pages). Comparing the AI reviewer's scores against my own hand-scored ratings gave a high agreement score ($\kappa = 0.896$).

4. The Five Biggest Ways It Gets Things Wrong

1. **Sarcasm/cursing can hide a real emergency.** "*@user you actually suck... I just bought a cab for a very drunk girl. She was sick out window...*" — This was a real safety situation, but the swearing and tone drowned out the actual warning signs, so it got read as general venting. *Fix:* check specifically for danger words ("drunk," "sick," "swerved") even inside angry messages.
2. **Messages about two things at once.** "*...driver cancelled the ride by himself why am I liable to pay cancellation charges due to driver unprofessional behaviour*" — This is both a rude-driver complaint and an unfair charge, but the system can only pick one category (it picked billing). *Fix:* let the system tag more than one category per message.
3. **Old broken links from years-old data.** A retrieved "similar case" included a 2017 shortened link that no longer works. *Fix:* strip old links from replies and swap in a current, working one.

4. **Legal threats mixed with billing complaints.** *"charged me \$15 for a ride I cancelled. If I do not get my refund today I will have my lawyer press charges."* — Correctly sent to a human instead of auto-resolving, since a bot shouldn't respond to legal threats. *Fix:* separate "lawsuit risk" from "legal threat attached to a billing complaint" so it reaches the right team.
5. **Rants with no real content.** *"Why does this always have to happen to me every single Tuesday???"* — Nothing to act on here, and the system correctly stayed silent. No fix needed — this is working as intended.

5. Being Honest About What the Headline Numbers Hide

1. Why the review-flagging rate looks "bad" (29.8%). The simpler search approach flagged things perfectly when it did (100%) — but missed half of all real emergencies. Missing an actual assault or crash could mean real harm; my system is deliberately tuned to send more things to a human rather than risk that. An unnecessary human check costs a few cents; a missed emergency could cost a life.

2. The test set doesn't reflect real-world traffic. I built the 180-message test set with roughly equal numbers per category (30 each) so rare-but-critical categories like safety get properly tested. In real traffic, safety incidents are under 0.2% of tweets while billing complaints are over half — so real-world performance would look somewhat different from this evenly-balanced test.

3. This only tests one message at a time. My evaluation looks at single messages and single replies, not a full back-and-forth conversation. A reply scored "3/5 — kind of generic" (like "check your Activity tab") might actually be exactly the right first step, even if it doesn't look impressive in a one-shot test.

Takeaway: A good benchmark score doesn't mean a system is production-ready. What matters most here is catching every safety issue and failing gracefully to a human — not chasing a higher accuracy number.

6. What I'd Build Next, One Week at a Time

Day	Focus	What I'd Build
1	Stronger safety detection	Improve keyword checks to understand context, and test against attempts to confuse the system.
2	Multi-part complaints	Let the classifier tag more than one category at once.
3	Fixing broken links	Auto-replace old/dead links with current, working ones.
4	Remembering the conversation	Let the system read the whole thread, not just one tweet.
5	Connecting to real systems	Hook into internal tools (e.g. refund eligibility) so replies use real data.
6	Cheaper and faster	Train a smaller model to replace the larger one for writing replies.
7	Safe production testing	Run silently alongside the human team to see how often its suggestions would be accepted.

Key Decisions I Made and Why

#	Decision	Why	What I Skipped
1	Split data by conversation thread, not tweet	Avoids the system "cheating" by seeing parts of the same conversation in both training and testing	Random tweet-level split
2	Left out very long threads (10+ tweets)	Long threads tend to be messy arguments that confuse tone	Keeping all raw threads
3	Matched on the customer's message, not Uber's reply	Finds new complaints similar to <i>past complaints</i> , not just similar-sounding replies	Matching against past replies
4	Lightweight search + classifier, not a heavy model	Runs in under 10ms with solid accuracy on messy tweets	Fine-tuning a large model just for sorting
5	Two-layer safety check (keywords, then AI backup)	Guarantees nothing dangerous slips through	Relying only on the AI
6	Three routing tiers, not a simple yes/no	Matches how real support teams prioritize their queue	A binary auto-reply/don't system
7	Built in a "stay silent" option	Prevents posting something wrong to a vague rant	Trying to reply to everything
8	Draft suggestions for human review	Speeds up how fast a human can approve a response	Leaving flagged messages with no draft
9	Replaced old shortlinks with in-app instructions	2017-era Twitter links are dead and lead to broken pages; in-app paths like "Activity > Find lost item" never expire and give real steps	Copying past Uber links word-for-word
10	Picked 30 cases per category instead of random sampling	Real safety emergencies are under 0.2% of tweets — a random sample would likely contain zero, making it impossible to test how well the system catches them	Pure random sampling from the test set
11	Kept legal threats separate from physical danger	A customer threatening a lawyer over a \$15 fee needs billing/legal review, not an emergency safety handoff	Treating all angry legal language as a physical safety emergency